

Professional Experience

📁 My professional journey so far — two full-time roles, internships, a co-founded startup and freelance engagements, newest first. Each entry notes the stack and what I owned.

Roles at a glance

Role	Organisation	Type	Period
Software Engineer	Mark AI	FULL-TIME · CURRENT	Jun 2026 – Present
Full Stack & AI Engineer	ESG Ratings	FREELANCE · CURRENT	Jul 2026 – Present
Software Engineer	VibeMonitor	FULL-TIME	Sep 2025 – Jun 2026
Full Stack Developer	Sakhi Women	FREELANCE	Jan – Feb 2026
Full Stack Developer	Mark AI	FREELANCE	Dec 2025 – Jan 2026
Full Stack & Server Developer	iTax Easy	INTERNSHIP	Feb – Sep 2025
Co-Founder & Technical Lead	PrepSaarthi	STARTUP	Nov 2024 – Jul 2025
Frontend Engineer → Mentor	AI-Zira Technology	INTERNSHIP	Feb – Dec 2024

🟢 Mark AI — Software Engineer (FULL-TIME · CURRENT)

Jun 2026 – Present · AI-native DOOH (digital out-of-home) advertising platform — Mark AI owns and operates the physical screens

📖 Full breakdown on the dedicated page: **Mark AI — What I Built (Current Role)** — 20 themes, representative PRs and a month-by-month timeline.

Advertisers book ad slots that play on physical Android signage screens in cafés and venues. I am the **sole engineer** on the platform and own the stack end to end — the backend that is the fleet's source of truth, the Kotlin player and updater apps on every screen, the operator CMS (web and phone), the advertiser platform, and the AI / creative / 3D layers on top.



Evidence	Value
PRs authored on the current platform	319 (313 merged) across 10 repos
Commits since 15 Jun 2026	482 across 12 repos — sole engineer on all of them
Backend	27 domain modules · 84 Alembic migrations

Highlights

- **Built the signage backend from the first commit** — a domain-driven **FastAPI + PostgreSQL** service that is the fleet's single source of truth, and **removed the WebSocket** in favour of a stateless heartbeat contract (`register · heartbeat · schedule · proof`). No held sockets means the fleet scales horizontally.
- **Built the Kotlin Android player** that runs fullscreen 24/7 on physical boxes with no Google Play Services — zero-touch enrolment from a key baked into a per-owner build, re-registration when its token expires, operating-hours gating with an off-hours keep-alive, and location even on an Ethernet-only screen. It keeps playing cached ads after a reboot with no network, and ships for two Android floors (Android 5 and Android 7+) from one codebase.
- **Built the Updater app — a second APK that keeps the Player updated** without anyone touching the box. It runs the Tailscale engine inside itself and joins our Headscale network with a join key stamped into its download, checks in with the backend, installs updates **silently as Android Device Owner** (after verifying sha256, package and version code), and is watched through a health window: an unhealthy build is **rolled back automatically** by rebuilding the last good release under a higher version code. Player and Updater share one signing key, enforced by the release pipeline.
- **Made proof-of-play provable** — plays attributed by `booking_id`, milestone screenshots armed on the playable timeline, captures fired on the ad's render callback (never a timer, never a half-rendered layout, never the same milestone twice), and a GPS overlay burned into each shot. A play is billed only when the ad renders and its turn completes; a **trusted server clock** back-dates offline plays so a box with a wrong clock cannot produce wrong billing; and a daily delivery ledger shows expected vs delivered. This is the billing trail.
- **Shipped an AI co-pilot** that books campaigns from plain English — tool-calling over the live domain, two guardrail layers, a deterministic slot-capture backstop so it never re-asks, per-screenowner scoping, and proof-of-play gap analysis. It also works over **WhatsApp** (linked handsets, booking slips, milestone photos) and turns advertiser **Gmail** messages into bookings behind a review step. Evaluated with an actor/verifier agent harness against real chat sessions.
- **Wrote the location-matching engine** behind "screens near X" — a typo-tolerant word-coverage scorer (ported to the frontend and guarded by a shared golden table), haversine distance, forward geocoding, a padded fleet bbox and locality misspelling correction.
- **Built the CMS as a phone app** — Kotlin Compose Multiplatform for Android and iOS from one codebase: scheduling, booking and editing campaigns, placing an ad on the panel, Creative Fit upload and proof-of-play review.
- **Built the advertiser platform** (Next.js 15 + a second FastAPI service) — marketing site with a 3D city hero, email-first auth, booking wizard, a live slot map, proof-of-play reports and a screenowner console. It plugs into the same fleet through the `/api/ext/*` partner API rather than a second source of truth.
- **Owned the commercial surface** — the four-part booking-approval workflow, **Razorpay** payments, per-key API attribution for partner bookings, campaign **PDF** reports and full proof-of-play **Excel** exports.
- **Screen hardware health** — integrated NovaStar's **VNNOX Care** API so the CMS shows the LED panel's real state (cabinet size, module type, dead cabinets), and dead cabinets are painted as dark patches on proof screenshots.

- **Ran it on GCP** — Cloud Run in front of Cloud SQL, Workload Identity Federation instead of service-account keys, signed URLs instead of public buckets, direct-to-GCS uploads, a secrets repo whose CI applies service config to Cloud Run on push to `main`, and a read-only developer dashboard that prices every billable resource.
- **Creative Fit, Studio, 3D & AR** — refit any finished ad, image or video, to a screen's real resolution (Gemini blueprint + LaMa inpainting + Pillow relay layout, output kept native/4K); split it out as **Mark AI Studio**, a public product with its own FastAPI service, job worker pool and API keys; a self-hosted **Hunyuan3D** generator on a Cloud Run L4 GPU; and an AR showroom for previewing a campaign in a real space.

● **ESG Ratings — Full Stack & AI Engineer (FREELANCE · CURRENT)**

Jul 2026 – Present · AI-driven ESG rating platform (esgratings.co.in) — sustainability scoring for corporates and BFSI lenders

Repositories: `akshatgg/esgrating-back` (Python) and `akshatgg/esgrating-web` (TypeScript).

- **Rebuilt the whole platform** — replaced the legacy PHP calculators and MySQL data with one **FastAPI service on MongoDB** (Python 3.12, uv), and rebuilt the public site, both calculators (ESG and BFSI) and the superadmin dashboard in **Next.js 16 (App Router), React 19, TypeScript and Tailwind CSS v4**.
- **Ported both scoring pipelines with exact parity** to the old system, down to PHP's round-half-away-from-zero rounding and cast behaviour, locked with tests. A one-off script migrated the MySQL data into MongoDB.
- **Report reading** — PDF and DOCX ingestion, a `pdffonts` check and cached Tesseract OCR for scanned reports, with subprocess timeouts so a bad file cannot hang a job. Analyses run as background jobs.
- **Multiple AI agents, one per job** — a page classifier, a KPI evidence-scoring engine, report narrative and rating-driver writers, and separate BFSI category, keyword and qualitative agents, fanned out across report pages with a thread pool and SHA-256 caching so a repeat analysis costs nothing.
- **Scoring formula** — every KPI scored 0–100 on the evidence found, rolled up by the methodology's weights (35/30/35 E/S/G; BFSI weighted by loan type, with a lending recommendation) to a final grade. Adopted the client's CFC methodology and prompts and imported the Question for every KPI.
- **Switchable model provider** — OpenAI or **Amazon Bedrock** (Converse API), chosen from the dashboard, including which Bedrock model is used.
- **Sector lookup** — finds the company's official stock-exchange sector, verified and sourced.
- **Editable reports** — analysts edit KPI scores, reasons, pillar weights and the written rating in place; the server recomputes and one KPI edit reaches every report. Every claim cites the printed page number.
- **Outputs** — a detailed report and one-pager as PDF, and a client-branded **ESG Rating Summary (.docx)** with AI-written narrative, strengths, weaknesses and rationale. Reports are emailed from the dashboard.
- **Admin and site** — superadmin login (bcrypt compatible with the old hashes, signed session cookie, logout), ratings CRUD and CSV import, blog with a TipTap editor, Chart.js score charts, CSP and security headers, SEO metadata.
- **Deployed on AWS** — Lightsail, MongoDB Atlas over IAM authentication, secrets in SSM Parameter Store, Docker Compose and Caddy auto-TLS behind a one-command deploy script.

- Tests run fully offline — MongoDB faked with mongomock and no real model calls.

🟡 VibeMonitor — Software Engineer (FULL-TIME)

Sep 2025 – Jun 2026 · AI-driven observability platform

See the dedicated "**VibeMonitor — What I Built**" page for the full breakdown. In short: **244 PRs** across backend (Python/FastAPI) and frontend (Next.js/React) — owning observability integrations, the agentic RCA engine, Stripe billing, the auth system, log ingestion and the **Beryl** AI test-automation product.

🟡 Sakhi Women — Full Stack Developer (Freelance)

Jan – Feb 2026 · Women-only ride-booking platform

- Built the platform end-to-end with **Next.js, Express.js & MongoDB** — role-based user & admin dashboards and the full ride lifecycle (accept/decline, driver assignment, live status).
- WhatsApp OTP login & templated notifications on the **Twilio WhatsApp API**; cron-based recurring scheduling for subscription rides.
- Designed MongoDB indexes for ~70% lower query latency under 1,500–2,000 concurrent users.
- Built the pricing engine (per-km one-way/return/round, night-time 1.5× dynamic pricing), subscriptions, coupons & payment-gateway integration.

🟡 Mark AI — Full Stack Developer (Freelance, earlier engagement)

Dec 2025 – Jan 2026 · Ad-booking platform for digital out-of-home (DOOH) displays

This was the **first, freelance engagement** with Mark AI, on the earlier MongoDB + third-party-CMS product. I re-joined **full-time in June 2026** to build the current platform from scratch — see the Mark AI section at the top of this page.

- Built the platform end-to-end — **Next.js frontend, FastAPI backend, MongoDB** — with role-based dashboards and a tested booking flow.
- Integrated a third-party **Xibo-based signage CMS** (ScreenOx) portal & REST API for real-time screen-inventory management across 100+ displays — screen sync by Xibo display ID, layout/schedule mapping and proof-of-play statistics; Razorpay payments + a coupon system.
- Added server-side search & pagination across screens, users and bookings, tiered pricing with a discount breakdown, campaign recurrence settings, Google OAuth registration, and role-aware responses that hide sensitive booking data.
- Configured a round-robin load balancer across multiple FastAPI instances with connection pooling & async handling, plus MongoDB indexing for concurrent load; later moved the deployment onto Cloud Run.

🟡 iTax Easy — Full Stack & Server Developer (Internship)

📌 **Note:** iTax Easy is a **company I worked for** — *not* a personal project. Its code lives across several repos on my GitHub (`itax_next_main`, `Itax_insurance`, `Itax_insurance_app`, `Itax_APK`, `policy_itax`), but this is **professional / internship work**, so it belongs here under Experience and is deliberately kept out of the Personal Projects section.

Feb – Sep 2025 · Tax-filing platform (GST & ITR) · web + mobile

- Built high-performance web & mobile apps with **PostgreSQL, Next.js, Prisma & Android Studio**; integrated the Sandbox API for GST/ITR filing — cut load times ~45% and lifted retention ~30%.
- Redesigned the database architecture — ~60% faster query responses and ~30% lower server cost.
- Owned role-based dashboards (super-admin / admin / user), and hosting, deployment & load-balancing on hPanel for production traffic.
- Built an eLibrary module that aggregates publicly available government-portal records.

PrepSaarathi — Co-Founder & Technical Lead

Nov 2024 – Jul 2025 · Mentor-driven edtech startup

- Co-founded Prepsaarathi.com, owned the full technical roadmap, and generated **₹1 Lakh** in revenue.
- Built the entire frontend & backend — 5+ core features with **React.js + MongoDB** (Express, Mongoose, Socket.io, Razorpay, JWT) — then migrated the frontend to **Vite**, lifting engagement ~40%.
- Built a counsellor booking system (student-mentor scheduling, ratings & reviews) and a payment flow with automatic commission split.
- Implemented SMS-OTP auth, role-based dashboards, secure REST APIs; deployed on Hostinger (hPanel).

Al-Zira Technology — Frontend Engineer → Mentor (Internship)

Feb – Dec 2024 · Service-based software company

- Delivered multiple client-facing frontend projects in **React**, building responsive, interactive UIs.
- Engineered complex, smooth UI animations & motion effects for polished experiences.
- **Promoted from Engineer to Mentor** after ~5–6 months — guiding and reviewing other frontend engineers' work.

Education & Leadership

- **B.Tech, Computer Science & Engineering (AI)** — KIET Group of Institutions (AKTU), GPA 7.8/10, Nov 2022 – May 2026.
- **Tech Lead, DevOps Club** (KIET) — guiding members on DevOps, CI/CD & cloud practices.
- Member of **FOSSCU** (open-source community).